the option *Pipeline Syntax* (as shown in Figure 1), which can be accessed from the Jenkins project's landing page, if the project is set up using *Pipeline-style*. This will launch a new tab/window, as per the browser's configuration, providing the dropdown list of the various steps available, along with the options and a small text box to display the generated code snippet (as seen in Figure 2) for the selected step.

Here is a snippet of Pipeline as Code:

Figure 1: Pipeline syntax

Figure 2: Snippet generator

Figure 3: *Script* path

```
#!groovy
node('label') {
  stage('Prep') {
    git url:'https://github.com/your-name/your-repo-path.
git', branch: 'master'
  }
}
```

It's a good practice to start the script with the shebang header *#!groovy*, which helps in code highlighting. Some of the best practices in writing Jenkins Pipeline are listed at *https://dzone.com/articles/top-10-best-practices-for-jenkins-pipeline?fromrel=true*. The *'node' block* helps to group the various stages, and execute the steps defined in the stages in the build agents that have the 'label'. *stage block* groups different individual steps, which are executed on the build agent. A few examples of the commonly used steps are *git*, *sh* and *echo*.

Single-line comments are preceded by two forward slashes (//), and comments spanning multiple lines start with /* and end with */.

```
/*

This is a multi-line comment to highlight its usage and it
ends here.

*/
```

Build flow

Let's construct an example of a scripted pipeline to: i) prepare the workspace and download the source code, ii) compile the source code (a small program written in C language), and iii) archive the artifacts. These tasks will be codified using scripted Pipeline syntax. For the purposes of this discussion, the Jenkins project has been created and named 'OSFY-pipeline demo'.

Jenkinsfile in the link, *https://github.com/mramanathan/osfy-pipeline-demo/blob/master/pipeline_bin/Jenkinsfile.*, is the complete source (written in Groovy) for this demo project.

Though the Jenkins documentation mandates that this file be named *Jenkinsfile* and placed at the root of the repository, in our example, it is named *Jenkinsfile.groovy* and kept under the folder *pipeline_bin*. This convention of placing the Jenkinsfile at the *root* of the *repo* should be followed while setting up the project configuration in Jenkins.

A clean start

To ensure that every trigger of the OSFY project starts a fresh build job with a clean workspace, we use the step *deleteDir()*. While *node* and *stage* blocks are covered, a new step named *dir* is introduced here, and *deleteDir* is wrapped inside this step.

```
node('linux') {
   stage('Clean Start') {
     dir('demo-sources') {
       deleteDir()
     }
   }
}
```

The contents of the folder, *demo-sources*, if present, will be removed. In the subsequent stages, the same folder will be reused to maintain continued access to the downloaded source code.

Download the source code

The *git* step references the *master* branch to download source code from the specified GitHub repo that's referenced using the URL. This step can include other options, like using the credentials which may or may not be different from that set-up in the Jenkins master. The contents of the repo are downloaded and stored in *demo-sources*.

```
stage('Prepare Workspace') {
   dir('demo-sources') {
     git url: 'https://github.com/mramanathan/osfy-pipeline-
demo.git', branch: 'master'
   }
}
```